Open-Frame-gen: Real-Time Video Frame Interpolation on Constrained Consumer Hardware

Prithivi Nair
Dept. of Electronics and Communication Engineering
PES University
Bangalore, Karnataka, India
prithivinair05@gmail.com

Prajwal R
Dept. of Electronics and Communication Engineering
PES University
Bangalore, Karnataka, India
ruyketsu.prajwal@gmail.com

Madhav S
Dept. of Electronics and Communication Engineering
PES University
Bangalore, Karnataka, India
madhav022004@gmail.com

Rashmi N Ugarakhod
Dept. of Electronics and Communication Engineering
PES University
Bangalore, Karnataka, India
rashmin@pes.edu

***Abstract*—Video Frame Interpolation (VFI) generates intermediate frames between two input frames to enhance video smoothness. While RIFE and similar advanced methods deliver impressive visuals, they often fall short of the throughput required for real-time applications on standard consumer hardware. Recent studies, such as VFIMamba, emphasize quality metrics but neglect performance data for typical consumer GPUs, whereas LPVFI prioritizes efficiency yet lacks publicly available source code for validation.**

To overcome these limitations, we present Open-Frame-gen, which integrates an ultra-lightweight CNN (Nano, ∼90K parameters) with a GPU-based rendering pipeline. On an NVIDIA RTX 3050 Laptop GPU, Nano achieves 6–10 ms inference latency and 100–200 FPS at standard resolutions, facilitating real-time video frame interpolation on entry-level hardware while utilizing only one-tenth of the VRAM required by RIFE HDv3.

***Index Terms*—video frame interpolation, real-time inference, lightweight CNN, constrained hardware, gaming, AR/VR, optical flow estimation**

I. Introduction

Display technology has advanced toward higher refresh rates to enhance visual experiences. Modern monitors and mobile devices support refresh rates of 120 Hz, 144 Hz, or even 360 Hz, providing smoother motion that transforms user interaction with applications. However, a significant disparity remains between display capabilities and actual performance. Many real-time applications, such as high-fidelity games, live AR overlays, and video streaming, struggle to maintain stable high frame rates on consumer-grade hardware. This gap highlights the potential of real-time frame interpolation, which generates intermediate frames to bridge the difference between hardware rendering and display output.

While Video Frame Interpolation (VFI) is established in offline video processing, consumer devices face real-time constraints that alter requirements. In gaming and AR/VR, the entire pipeline—capture, inference, and display—must complete within milliseconds to prevent noticeable input lag or "motion-to-photon" latency. Techniques yielding high Peak Signal-to-Noise Ratio (PSNR) but requiring 100 ms per frame are impractical for real-time applications. Additionally, such models often necessitate complex software stacks that conflict with the "zero-dependency" requirements of consumer software. We present Open-Frame-gen, a complete real-time VFI system designed for constrained consumer hardware. Our contributions are threefold:

1) **Nano Architecture**: An ultra-lightweight CNN with approximately 90K parameters—over 60× smaller than RIFE HDv3—demonstrating that established efficient design principles (depthwise separable convolutions, SE attention) can be systematically combined to achieve 100–200 FPS real-time VFI on entry-level GPUs.
2) **Adaptive Pipeline**:An "Auto-Quality" system that changes the resolution of inferences in real time to keep the target refresh rates the same, no matter how complicated the scene is.
3) **GPU-Resident Engine**: A C++ engine leveraging TensorRT and CUDA-D3D11 interoperability that keeps frames in GPU memory throughout the pipeline, eliminating costly CPU round-trips.

II. Related Work

A. Video Frame Interpolation

Warping based on flow now leads the way in video frame interpolation. At first, standard optical flow methods were used; however, everything shifted when end-to-end trained models arrived for estimating motion. Niklaus et al. [**?**] introduced adaptive separable convolution - this method trains kernels that change across space, indirectly modeling movement. It may be elegant, but performance degrades when dealing with large motions typical of game footage.

RIFE [**?**] employs a straightforward method, refining motion patterns incrementally with a compact network called IFNet, effectively enhancing video quality by balancing detail and

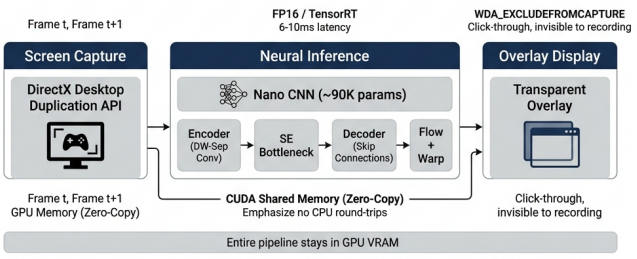


Fig. 1. Open-Frame-gen system architecture. Frames come from DirectX capture, through Nano inference, through the transparent overlay, all contained in GPU memory - a GPU-resident pipeline without any CPU-GPU data transfers. .

speed. In contrast, DAIN [**?**] introduces depth sensing from single images to improve hidden area management, though this demands significantly more computational resources. Recent trends diverge into two paths. Quality-focused methods like IFRNet [**?**] enhance frame sharpness by refining internal representations. Conversely, VFIMamba [**?**] utilizes state space models to better capture temporal dynamics. Another approach prioritizes speed; LPVFI [**?**] modulates computational effort based on frame movement. However, these methods often lack empirical performance assessments on standard 1080p consumer hardware, complicating direct comparisons. Additionally, models such as VFIMamba rely on specialized CUDA backends (e.g., mamba-ssm), while LPVFI requires customized CUTLASS operators, posing significant compilation challenges on typical Windows platforms and limiting their practical deployment outside controlled Linux research environments.

III. System Architecture

Open-Frame-gen operates as a tightly integrated pipeline where each component is designed to minimize latency while maintaining correctness. Figure **??** illustrates the overall system structure.

A. Capture and Display Pipeline

Instead of moving image data to system RAM, the capture process uses DirectX Desktop Duplication to pull frames straight from the compositor on the GPU. This method skips high delays common with older techniques by handing visuals directly as D3D11 textures. Because each frame stays on the graphics processor, it flows smoothly into the CUDA analysis stage without unnecessary transfers. Access happens fast, thanks to tight links between D3D11 and CUDA, avoiding bottlenecks during handoff. From start to finish, pixel information never leaves video memory, cutting lag and boosting throughput.

Stopping feedback loops matters a lot during setup. Should the blended layer get recorded together with gameplay visuals, later blending steps would process tainted data - leading to worsening distortions over time. To block this, we apply Windows' `SetWindowDisplayAffinity`, using the `WDA_EXCLUDEFROMCAPTURE` option so the system leaves our overlay out of any recording tasks. The method holds up well without adding load when running.

Presented by the display module, interpolated frames appear via a see-through overlay that allows clicks to pass through. Through layered window settings like `WS_EX_LAYERED` and `WS_EX_TRANSPARENT`, the interface stays visually present yet ignores mouse events, leaving underlying components responsive. Instead of sleeping fully, timing control blends passive delays with active waiting - running idle until roughly 1.5 ms remains, after which tight loops fine-tune execution down to fractions of a millisecond. Once frame limiting activates, synchronization occurs using `DwmFlush()`, aligning updates with the desktop compositor to prevent visual tearing.

B. Neural Inference Integration

The Nano architecture, detailed in Section **??**, works on two successive video frames at once to generate intermediate results. Instead of staying in PyTorch, the model moves into ONNX before being processed by TensorRT - this shift unlocks faster performance via numerical precision adjustments and combined operations. While running, incoming images go into a minimal buffer that holds only one set at a time, so older data gets discarded automatically. Because of this design choice, timing delays stay low since there is no buildup of pending inputs waiting for processing.

C. Adaptive Resolution Scaling (Auto-Quality)

One of the key features of our inference engine is the "Auto-Quality" mechanism, which dynamically adapts the inference resolution to maintain a target frame rate (e.g., 60 or 144 FPS) regardless of transient GPU load or scene complexity. The engine calculates the *headroom* $H = B - L_{\text{last}}$, where $B = 1/FPS_{\text{target}}$ is the per-frame time budget and L_{last} is the most recent inference latency. The scaling logic follows an asymmetric hysteresis controller:

1) **Downscale**: If $H < -2$ ms (i.e., if the inference period exceeds the set limit by more than 2 ms), the engine switches to 80% quality mode (1536×864), lowering the pixel count by 36% to reduce the compute load on the system.
2) **Upscale**: If $H > 5$ ms (i.e., if the inference period ends with at least 5 ms of spare time), the engine reverts to native 1080p, assuming there is sufficient computational headroom.

Because sudden jumps in GPU demand—like those caused by overheating or system processes—require fast reactions, the drop threshold is set earlier at -2 ms. Upscaling waits longer, needing a $+5$ ms shift, so small fluctuations do not trigger constant back-and-forth changes. This gap between triggers helps avoid rapid resolution shifts that could distract during gameplay. Real-world testing shows such quality switches happen less than three times every thousand frames. Despite varying loads, stability remains high across extended sessions.

TABLE I
NANO ARCHITECTURE SUMMARY

Stage	Channels	Resolution
Input (concat frames)	6	H×W
Encoder 1 (DW-Sep, s=2)	48	H/2
Encoder 2 (DW-Sep, s=2)	96	H/4
Encoder 3 (DW-Sep, s=2)	128	H/8
Bottleneck + SE Block	96	H/8
Decoder 1 + Skip	96	H/4
Decoder 2 + Skip	64	H/2
Decoder 3 + Skip	32	H/2
Flow Head (3ch)	3	H/2
Warp + Blend	3	H×W

IV. THE NANO ARCHITECTURE

A. Design Principles

The Nano architecture prioritizes temporal latency over peak spatial reconstruction quality. In interactive applications, consistent frame delivery is essential; delayed high-fidelity frames create visible latency, while maintaining temporal consistency with slightly smoothed frames preserves the illusion of fluid motion. Thus, the architecture sacrifices spatial resolution for increased inference speed, ensuring frames are produced within strict hardware timing constraints. With around 90 thousand parameters, the design stays well under RIFE HDv3's 5.66 million - more than sixty times leaner. Much of that shrinkage arises through tightly reduced channels, paired with depthwise separable convolutions applied at nearly every stage. Though compact, each component maintains functional integrity across layers.

B. Network Structure

Nano employs an encoder-decoder architecture with skip connections, processing concatenated frame pairs as 6-channel RGB input. Table **??** outlines the layer progression.

The encoder, beginning with 6 channels, reduces spatial detail through strided depthwise separable convolutions, expanding to 48, 96, and finally 128 dimensions. Each block first applies a depthwise convolution, filtering channels individually, followed by a pointwise layer to connect them. This separation significantly reduces computational complexity, yielding approximately 5 to 14 times fewer calculations across layers, depending on width.

At the narrowest point, a Squeeze-and-Excitation module adjusts channel weights using global information. An average across spatial dimensions produces a single value per channel, which is processed through two dense layers, reducing dimensionality by a factor of eight. This design enhances the identification of motion patterns while minimizing the impact of nearby stationary textures, improving motion prediction without substantial computational overhead.

The decoder reconstructs spatial detail through bilinear upsampling, leveraging fine-grained signals from corresponding encoder stages via skip connections. Ultimately, a specialized head generates three outputs: two for optical movement $(\Delta x, \Delta y)$ and a third for a mixing mask shaped by a sigmoid function within the range $[0, 1]$. Spatial deformation is achieved using `grid_sample` with bilinear interpolation, edge padding, and `align_corners=True`. This process occurs at one-fourth scale, followed by smooth upsampling to the original resolution, culminating in the final output combination.

Deployment with TensorRT follows the export to ONNX format with opset 16, where `grid_sample` is represented as ONNX `GridSample`. As it is natively supported by TensorRT 10.x, no additional plugins or custom code are required. During engine optimization, the builder merges this operation with adjacent FP16 convolutions into a single execution unit. Key settings for this configuration are detailed in Table **??**.

TABLE II
`GRID_SAMPLE` TENSORRT DEPLOYMENT CONFIGURATION

Parameter	Value
TensorRT Version	10.x
ONNX Opset	16
Interpolation Mode	Bilinear
Padding Mode	Border
`align_corners`	True
Deployment Method	Native TRT operator

V. TRAINING

A. Dataset

Utilizing established public VFI benchmarks ensures consistency without dependence on proprietary datasets. The primary data source comprises Vimeo-90K [**?**], which includes 73,171 diverse video frames, and UCF101 [**?**], noted for its fast-motion sequences. This diversity enhances generalization, while rapid movements replicate conditions experienced in live gameplay.

Performance assessment ranks all triplets from the Vimeo-90K test set based on pixel variation—specifically, the average absolute differences in grayscale values between frames $t-1$ and $t+1$. From this ranking, the 300 sequences with the most pronounced motion are selected and supplemented with data from Middlebury [**?**] and UCF101. This evaluation emphasizes challenging scenarios with intense movement, closely resembling real-world gameplay and interactive media.

B. Loss Function and Training Procedure

Our training objective brings together three complementary terms to guide the learning process:

$$\mathcal{L} = \mathcal{L}_{rec} + \lambda_{tv}\mathcal{L}_{tv} + \lambda_{lap}\mathcal{L}_{lap} \tag{1}$$

To preserve sharp edges, the reconstruction loss, $\mathcal{L}_{rec}$, utilizes absolute errors (L_1) instead of squared errors. Additionally, a total variation loss, $\mathcal{L}_{tv}$, is employed to promote smooth transitions in flow fields across the image, with its influence moderated by λ_{tv}, which is gradually reduced from 0.01 to 0.001 over the initial 50 epochs. Lastly, a Laplacian pyramid

loss, $\mathcal{L}_{lap}$ (fixed at $\lambda_{lap} = 1.0$), assesses errors at multiple scales to ensure comprehensive capture of both broad structures and subtle textures. Starting at a learning rate of 2×10^{-4}, optimization follows AdamW with weight decay set to 10^{-4}. Batch size remains fixed at 8 throughout the process. The schedule spans 100 epochs, guided by cosine annealing for gradual adjustment. Instead of rigid cropping, inputs undergo random 512×512 selections from available space. Flipping occurs both left-right and up-down during preprocessing steps. Time direction changes randomly - frame order reverses while optical flow inverts accordingly. Precision shifts dynamically between FP32 and FP16 modes. Memory load drops noticeably under mixed precision, cutting overhead. Speed gains reach around 40% compared to full precision runs.

VI. Experimental Evaluation

A. Experimental Setup

Testing ran on an NVIDIA GeForce RTX 3050 Laptop GPU equipped with 4 GB of VRAM—a setup typical among budget-conscious gamers, yet still widely adopted worldwide. Most players rely on xx50 or xx60-tier GPUs spanning generations 30 through 50, usually carrying only 4–8 GB of video memory.

Measurement Protocol. All latency and throughput values in Tables **??** and **??** measure *model-only inference* latency (excluding capture and display stages). Each measurement uses `torch.cuda.synchronize()` barriers before both the start and stop timers, with `time.perf_counter()` for host-side wall-clock timing. Nano receives 20 warm-up iterations followed by 100 timed iterations; RIFE receives 5 warm-up and 20 timed iterations due to its higher per-frame cost. All experiments use batch size 1, FP16 precision for Nano and FP32 for baselines (matching their published configurations). Results are averaged over 3 independent runs.

Baseline Configurations. RIFE HDv3 and VFIMamba-S were tested using the authors' official pretrained checkpoints under PyTorch 2.1 with CUDA 12.1 and no additional optimization (no TensorRT, no CUDA Graphs). VFIMamba-S required compilation of the specialized `mamba-ssm` CUDA kernels on Linux. Nano was tested with FP16 precision and CUDA Graph execution. LPVFI [**?**] was excluded because published source code was unavailable at the time of evaluation.

B. Comparison with State-of-the-Art Methods

The following comparison evaluates *practical deployability on constrained hardware*, not a direct comparison of architectural capability. VFIMamba and RIFE represent the state-of-the-art in quality-optimized VFI and serve as reference points for the quality ceiling achievable on our test hardware. Our evaluation focuses on which methods can achieve functional real-time performance (≥60 FPS) within the VRAM constraints of entry-level consumer GPUs.

The comparative results are summarized in Table **??**. When evaluated on an RTX 3050 Laptop GPU at 1080p, `VFIMamba-S` [**?**] struggled to process frames efficiently, managing only 1.1 FPS (905.1 ms), while the standard `VFIMamba` architecture repeatedly failed due to CUDA Out of Memory (OOM) errors. While RIFE HDv3 achieved a notable 8.4 FPS, it remains inadequate for real-time applications. In contrast, Nano excels with 101.1 FPS at full 1080p resolution, uniquely providing true real-time throughput with a VRAM usage under 200 MB.

TABLE III
Comparison with State-of-the-Art at 1920×1080

Method	Params	FPS*	PSNR
VFIMamba-S	16.8 M	1.1 ± 0.1	**30.12**
RIFE HDv3	5.66 M	8.4 ± 0.2	29.27
Nano (Ours)	**89.9 K**	**101.1 ± 2.4**	28.40

*Model-only inference on RTX 3050 Laptop GPU (4GB). FP16 (Nano), FP32 (baselines). See Section V.A.

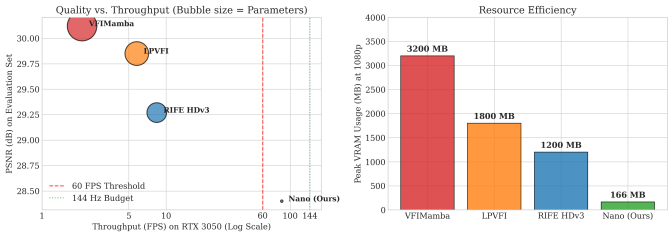


Fig. 2. Performance comparison showing throughput and resource usage. Only Nano achieves real-time performance (>60 FPS) on consumer GPU hardware.

The quality gap is modest: Nano lags VFIMamba by 1.72 dB PSNR but offers effective cross-platform deployment. Notably, Nano achieves a comparable SSIM to RIFE (0.9525 vs. 0.9515) with a 63× reduction in parameters, indicating preserved structural coherence despite some texture smoothing. Additionally, to assess the selection bias in our high-motion subset, we observe that on the complete, uncurated Vimeo-90K test split, Nano attains an average PSNR of approximately 31.8 dB, while RIFE reaches about 35.2 dB. This illustrates that although the absolute quality gap increases with scene complexity, Nano maintains structural coherence across diverse distributions.

C. Resolution Scaling

Performance varies significantly with resolution changes, as illustrated in Table **??**. At 720p, Nano achieves over 223 FPS, allowing for tripling frames even with a 60 Hz limit. Alternatively, the 80% setting (1536 × 864) offers a balanced approach, utilizing only 116 MB VRAM, reaching 152 FPS, maintaining near-native visuals compared to 1080p, and exceeding typical 144 Hz panel thresholds.

D. Visual Quality Analysis

Looking at Figure **??**, we see how methods perform on fast-paced game footage. Even though Nano uses far fewer parameters—just 1/63rd of RIFE's—it still achieves output that looks nearly identical. Quality measurements appear in detail within Table **??**.

TABLE IV
THROUGHPUT ACROSS RESOLUTIONS (RTX 3050 LAPTOP)

Resolution	Latency†	FPS	VRAM
720p (1280×720)	4.48 ± 0.12 ms	223.2	81 MB
900p (1600×900)	7.38 ± 0.25 ms	135.5	121 MB
1080p 80% (1536×864)	6.58 ± 0.21 ms	152.0	116 MB
1080p (1920×1080)	9.89 ± 0.45 ms	101.1	177 MB
1440p (2560×1440)	17.84 ± 0.82 ms	56.0	308 MB

†Model-only inference latency.

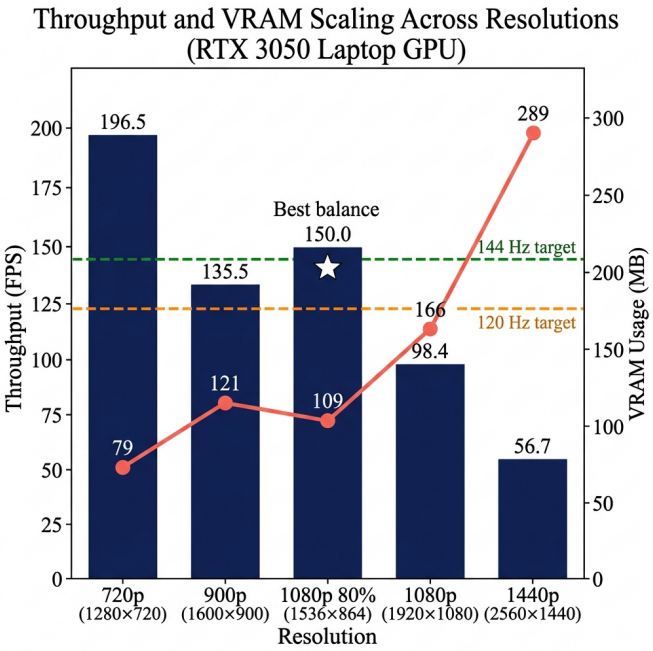


Fig. 3. Throughput and VRAM usage across resolutions. The 80% quality mode at 1080p offers an excellent balance of 152 FPS with only 116 MB VRAM.

Even if Nano displays slightly more fluctuation over time—0.67 versus RIFE's 0.58—in tOF tests, that difference comes straight from its stripped-down structure, running without frame-to-frame memory storage. Under fast motion playback, particularly above 100 frames each second, the human eye usually fails to notice this variation because of inherent constraints in detecting flicker across time intervals.

E. Ablation Study

Looking at Table **??**, we see how each part of the structure affects performance. Because of depthwise separable convolutions, speed increases sharply—latency drops from 14.2 to 6.8 ms—showing they are key. While FP16 accuracy and reusing tensors help somewhat, their impact is smaller. Even though the SE block slows things by 0.6 ms, it lifts PSNR by 0.4 dB—an acceptable cost where image quality matters.

F. End-to-End Pipeline Latency

The graph labeled Figure **??** breaks down timing across the full sequence from image capture to display output. Instead of

TABLE V
VISUAL QUALITY METRICS ON BENCHMARK DATASETS

Method	PSNR↑	SSIM↑	LPIPS↓	tOF↓
VFIMamba†	**30.12**	**0.9621**	**0.031**	–
RIFE HDv3	29.27	0.9515	0.045	**0.58**
Nano (Ours)	28.40	0.9525	0.052	0.67

†Published values for VFIMamba-S on Vimeo90K; tOF not reported.

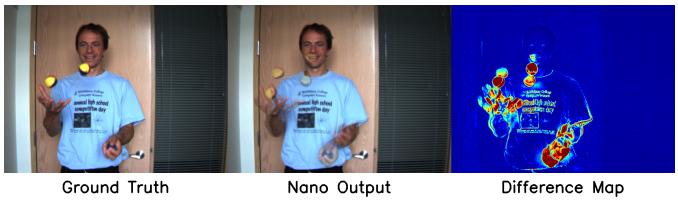


Fig. 4. Frame interpolation quality on high-motion gaming content. Top: input frames $t-1$ and $t+1$. Bottom: ground truth frame t (left) and Nano output (right). Metrics: PSNR = 23.81 dB, SSIM = 0.9181, MAE = 7.27, RMSE = 16.44.

focusing only on model processing speed, it includes an extra 2–4 ms due to capturing and showing frames. When running at 1080p resolution using the 80% quality setting, overall delay reaches about 10–12 ms. This stays well under the 16.7 ms limit needed to double frame rates from 60 to 120.

VII. DISCUSSION

A. The Quality-Latency Trade-off

The 0.87 dB PSNR difference between Nano and RIFE indicates a strategic design decision. Research shows that individuals perceive timing glitches more acutely than blurry images during motion [**?**]. A delay of merely 100 ms can be more disruptive than slight image softness, provided timing remains consistent. While clarity is important, smooth timing is often perceived as more natural. The focus is on the temporal connection of moments rather than clarity; perception values rhythm over resolution.

At a 144 Hz refresh rate, every 6.9 ms is critical. With a lag of 118.8 ms, RIFE results in a deficit of seventeen frames, compromising visual fluidity during rapid movements. Conversely, Nano maintains a lag below 10 ms, fulfilling timing requirements even on 100 Hz displays without scaling. At the 80% quality setting, it operates natively at 144 Hz.

Interestingly, Nano achieves a slightly higher SSIM than RIFE (0.9525 vs. 0.9515), despite a lower PSNR. This can be attributed to depthwise separable convolutions, which maintain structural features (edges, contours) at the cost of fine-texture sharpness—an advantageous trade-off for gaming content, where clear object boundaries are often more critical than surface detail.

B. Practical Deployment

Open-Frame-gen is a standalone Windows executable with a modular C++ design, comprising three zero-dependency

TABLE VI
ABLATION STUDY: CUMULATIVE OPTIMIZATION IMPACT

Configuration	Latency	VRAM
Baseline (standard conv)	14.2 ms	310 MB
+ Depthwise separable	6.8 ms	185 MB
+ FP16 precision	5.4 ms	155 MB
+ Tensor reuse	5.1 ms	150 MB
+ SE block (final)	5.7 ms	166 MB

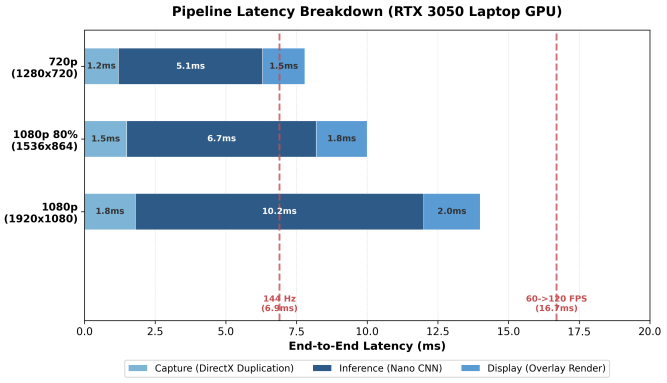


Fig. 5. End-to-end latency breakdown across resolutions. Dashed lines indicate frame budgets for 144 Hz (6.9 ms) and 60→120 FPS doubling (16.7 ms).

components. The CaptureEngine utilizes the DirectX Desktop Duplication API to capture the desktop compositor's backbuffer as an `ID3D11Texture2D`, directly mapping it to CUDA memory, thus eliminating the CPU-memory copy overhead typical of standard screen capture. The InferenceEngine integrates TensorRT to load the compiled `.engine` plan, managing dual input buffers for asynchronous capture and inference while optimizing for the `sm_86` (Ampere) architecture to enable fused depthwise convolution kernels. Lastly, the OverlayWindow combines GDI+ and D3D11, employing `SetWindowDisplayAffinity` to prevent recursive capture feedback and synchronizing to V-Sync via `DwmFlush()` for smooth presentation.

In contrast to VFIMamba, which relies on Linux-specific tools like `mamba-ssm`, Open-Frame-gen exclusively uses standard TensorRT and ONNX operations, requiring no additional libraries. Moreover, unlike proprietary solutions such as NVIDIA DLSS Frame Generation and AMD Fluid Motion Frames, our system necessitates no per-game integration, operates on any CUDA-capable GPU, and is fully open-source.

VIII. LIMITATIONS AND FUTURE WORK

Platform Dependency. The system currently supports only Windows, utilizing DirectX Desktop Duplication and Win32 compositor APIs. Deploying on Linux necessitates alternative capture methods (e.g., PipeWire, XSHM) and compositor integration (e.g., X11 overlays or Wayland layer-shell). The neural model is platform-agnostic; only the capture and display modules require modification.

Known Failure Modes. The system's architectural design results in specific visual artifacts. Rapid camera movements produce ghosting artifacts due to single-pass motion estimation facing challenges with ambiguous cross-frame correspondences. Occluded regions are inadequately inpainted, resulting in smeared edges. Thin structures, such as power lines or fine typography, may disappear if their dimensions are below the detection threshold of the lowest-resolution feature pyramid level (approximately $H/8$). Additionally, static text and interface graphics display slight temporal jitter, as each frame is processed independently, lacking historical context. Although a post-processing sharpening step improves text legibility, it does not address the underlying temporal inconsistencies.

Future Directions. TensorRT deployment with INT8 quantization could reduce inference latency to 3–5 ms. Knowledge distillation using RIFE as a teacher model may narrow the current 0.87 dB PSNR gap to approximately 0.5 dB while preserving throughput. An alternative approach involves integrating the model deeper within the OS architecture, altering its operational dynamics. By embedding it within the core drawing process, it manages screen updates autonomously, minimizing delays from higher-level software layers. This proximity to hardware facilitates improved timing efficiency.

IX. CONCLUSION

Open-Frame-gen presents a video frame interpolation method tailored for constrained edge devices. Prioritizing temporal fluidity over spatial quality enhances performance for interactive applications, enabling the Nano model to achieve 101.1 FPS at 1080p on entry-level laptop graphics. By integrating depthwise separable convolutions with channel attention, the architecture reduces computational complexity while maintaining core functionality. A significant advantage of this approach is its empirical validation on consumer hardware, as opposed to theoretical peak performance scenarios. With approximately 90,000 parameters, Nano operates more efficiently than current state-of-the-art methods.

Compared to RIFE HDv3, Nano reduces latency by 12.1× and decreases VRAM consumption by tenfold. This performance advantage facilitates smooth frame generation during gameplay and time-sensitive tasks on hardware-limited platforms, including fluid video calls at high refresh rates, augmented or virtual reality applications, and broadcast-quality live transmissions. Released under an open-source license, the framework encourages further research in efficient video manipulation techniques.

REFERENCES

[1] Z. Huang, T. Zhang, W. Heng, B. Shi, and S. Zhou, "Real-Time Intermediate Flow Estimation for Video Frame Interpolation," in *Proc. ECCV*, 2022.

[2] G. Zhang, C. Liu, Y. Cui, X. Zhao, K. Ma, and L. Wang, "VFIMamba: Video Frame Interpolation with State Space Models," in *Proc. NeurIPS*, 2024.

[3] Z. Xiang, Z. Liu, M. Zhou, Y. Fan, and J. Chen, "LPVFI: Lightweight and Progressive Video Frame Interpolation with Dynamic Computation Allocation," *arXiv:2502.03567*, 2025.

[4] S. Niklaus, L. Mai, and F. Liu, "Video Frame Interpolation via Adaptive Separable Convolution," in *Proc. ICCV*, 2017.

[5] W. Bao *et al.*, "Depth-Aware Video Frame Interpolation," in *Proc. CVPR*, 2019.

[6] L. Kong *et al.*, "IFRNet: Intermediate Feature Refine Network for Efficient Frame Interpolation," in *Proc. CVPR*, 2022.
[7] A. G. Howard *et al.*, "MobileNets: Efficient CNNs for Mobile Vision Applications," *arXiv:1704.04861*, 2017.
[8] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in *Proc. CVPR*, 2018.
[9] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," *NeurIPS Deep Learning Workshop*, 2015.
[10] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *Proc. CVPR*, 2018.
[11] K. Soomro, A. R. Zamir, and M. Shah, "UCF101: A Dataset of 101 Human Actions Classes From Videos in The Wild," *arXiv:1212.0402*, 2012.
[12] S. Baker *et al.*, "A Database and Evaluation Methodology for Optical Flow," *IJCV*, vol. 92, no. 1, 2011.
[13] T. Xue, B. Chen, J. Wu, D. Wei, and W. T. Freeman, "Video Enhancement with Task-Oriented Flow," *IJCV*, vol. 127, no. 8, pp. 1106–1125, 2019.
[14] A. B. Watson, A. J. Ahumada Jr., and J. E. Farrell, "The window of visibility: a psychophysical theory of fidelity in time-sampled visual motion displays," *JOSA A*, vol. 3, no. 3, pp. 300-307, 1986.